PROJECT REVIEW 2: SYSTEM ARCHITECTURE & IMPLEMENTATION REPORT

Project: Natural Language Mount & Partition Agent using LLM Tool Use (Gemini Function Calling)

NAME: SOUJANYA MUKHERJEE
REG NO: 24BCI0135

NAME: AKSHAT GUPTA
REG NO: 24BCE2930

Chapter 3: System Architecture & Design

In this phase of our project (Review 2), we designed and implemented the complete system architecture of the Mount & Partition Agent. On a standard Linux system, inspecting a disk image (such as an ISO) or a partitioned disk file requires root privileges and a multi-step sequence of commands (losetup, blkid, mkdir, mount). Beginners often make syntax errors or risk mounting the wrong device. To solve this safely, the agent accepts plain English requests and lets an LLM (Google Gemini) choose from a small, fixed set of typed tools. The model never receives a shell: every tool is a Python function that validates its own arguments and then runs a fixed command or a direct system call.

High-Level Architecture

Our system processes instructions through a closed five-stage feedback pipeline called the **User-to-Shell Loop**: **User Input → LLM Processing → Tool Call → System Execution → Feedback**. Because the model can only emit function calls that match registered schemas, raw natural language is never allowed to execute arbitrary shell strings.

- **1. User Input:** The user types a plain English instruction at the REPL prompt (e.g., *"Mount ~/Downloads/ubuntu-22.04.iso so I can explore the files"* or *"What's mounted right now?"*). The conversation history is kept so follow-up questions work.
- **2. LLM Processing:** The REPL sends the history to Gemini (`gemini-2.5-flash`, temperature 0) together with a system instruction describing the standard workflow and the nine tool declarations. The model decides which tool to call and with what parameters.
- **3. Tool Call:** The model returns one or more `function_call` parts. The dispatcher looks the tool name up in the `TOOLS_IMPL` registry and calls the matching Python function with the model-supplied arguments. Unknown tools or bad arguments are caught and returned as errors.
- **4. System Execution:** The tool validates its inputs (device allow-list, mount-name pattern, path confinement), asks for human confirmation if write access is requested, and then runs a fixed argv list (`lsblk, losetup, blkid, udevadm`) or calls libc `mount()` / `umount2()` through ctypes.
- **5. Feedback & Terminal Output:** The structured result (device, mountpoint, filesystem type, errors) is sent back to Gemini as a `FunctionResponse`. The loop repeats until the model stops calling tools, and its final text is printed to the terminal.

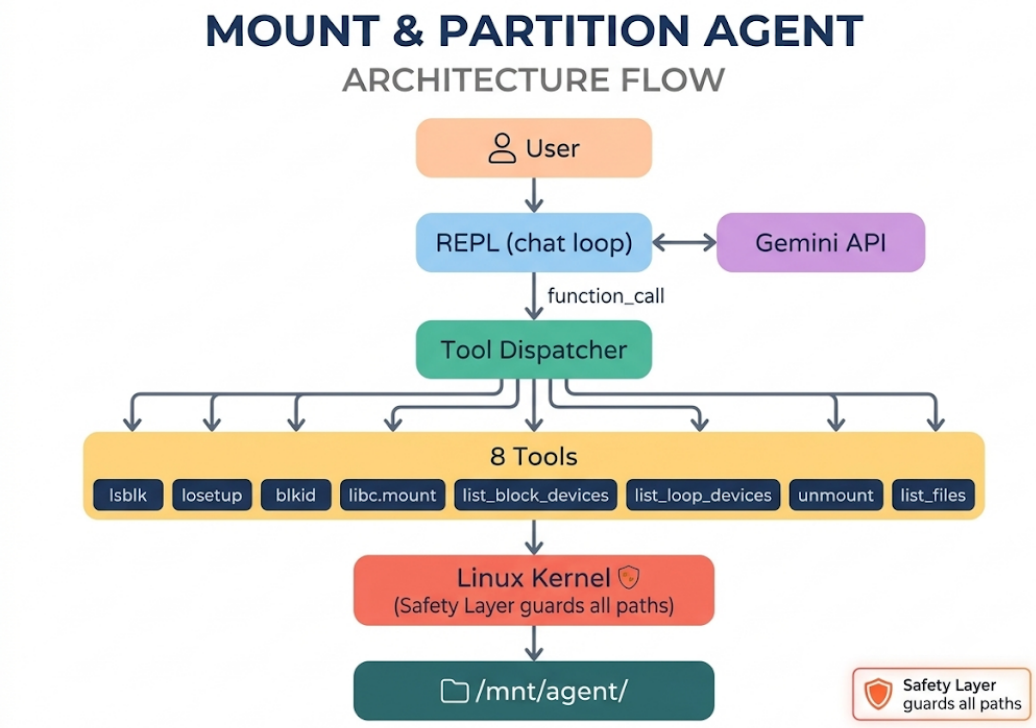


Figure 3.1: System Architecture Block Diagram — User, REPL, Gemini API, Tool Dispatcher, Tools, Linux Kernel and the /mnt/agent sandbox.

Operational Execution Flowchart

The complete decision logic of the agent is illustrated in Figure 3.2. It highlights the tool-call decision, the read-write confirmation gate (human-in-the-loop), and the feedback path that returns each result to the model.

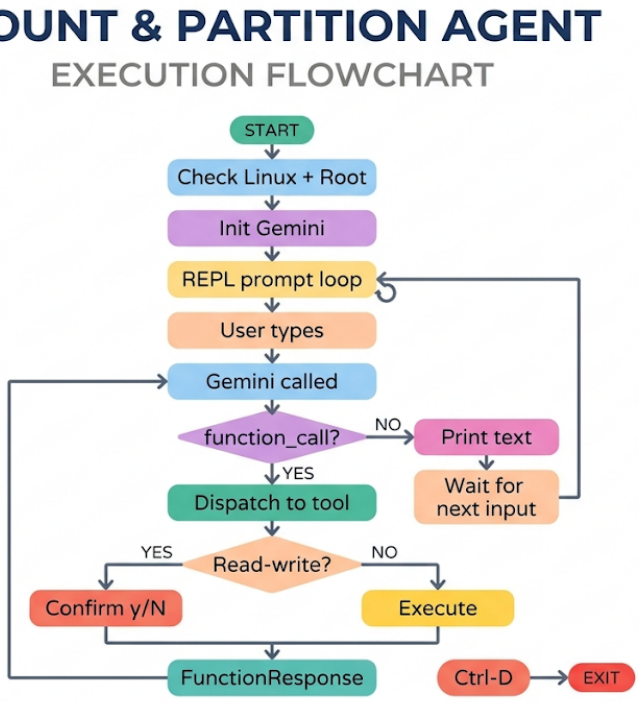


Figure 3.2: Detailed Execution Flowchart — From user input to Gemini tool call, safety confirmation, execution and FunctionResponse.

Component Design

We decomposed the project into three loosely coupled layers inside a single root-run process, so that language understanding is cleanly separated from system access:

- **1. The Interface Layer (REPL):** Implemented by `chat()` in `mount_agent.py`. It reads user input, keeps the conversation history, prints model text and `[tool]` trace lines, and exits cleanly on Ctrl-D / Ctrl-C.
- **2. The Orchestration Layer (LLM + Dispatcher):** The Gemini client, the `SYSTEM` instruction, the `TOOLS` function declarations and the agentic loop that executes every `function_call` through `TOOLS_IMPL`.
- **3. The Tooling Layer (Safe System Wrappers):** Nine tool functions plus helpers (`run, device_allowed, mount_dir, confirm`) that wrap lsblk, losetup, blkid and the libc mount/umount2 calls.

Layer / Module	Location	Privilege Level	Core Functionality
Interface Layer (REPL)	mount_agent.py: chat()	Root (sudo -E)	Prompt loop, conversation history, tool-trace output, graceful exit.
Orchestration Layer (LLM + Dispatcher)	mount_agent.py: SYSTEM, TOOLS, TOOLS_IMPL	Root (sudo -E)	Gemini function calling, tool-name dispatch, error wrapping, repeat until no more tool calls.
Tooling Layer (System Wrappers)	mount_agent.py: 9 tools + helpers	Root (needed for losetup / mount)	Fixed-argv subprocess calls, libc mount()/umount2() via ctypes, input validation.
Setup & Demo Scripts	setup.sh, demo.sh, requirements.txt	Standard user	Python 3.10+ check, dependency install, system-tool check, API-key check, example prompts.

Registered Tools

Tool	Underlying Primitive	Purpose
list_block_devices	lsblk -J	List disks, partitions, filesystems and mountpoints as JSON.
list_loop_devices	losetup -J -l	Show active loop devices and the image files backing them.
attach_loop	losetup --find --show --partscan	Attach an ISO / disk image to a free loop device (read-only by default).
detect_filesystem	blkid -p -o export	Identify filesystem TYPE, LABEL and UUID of a device or partition.
mount_device	libc mount(2)	Mount a device at /mnt/agent/<name>; read-only by default, type auto-detected.
list_mounts	/proc/self/mountinfo	List mounts the agent made under /mnt/agent.
unmount	libc umount2(2) + losetup -d	Unmount, remove the directory and detach the backing loop device.
detach_loop_device	losetup -d	Clean up a loop device left over after a failed mount.
list_files	pathlib (confined)	Browse a directory inside a mount without escaping the mount point.

Security & Guardrail Framework

A major concern with an AI agent that runs as root is that a hallucinated or malicious prompt could damage the host system. To address this, the agent enforces several interlocking guardrails in code, so they hold regardless of what the model asks for.

A. The Human-in-the-Loop (HITL) Confirmation Mechanism

Operations fall into three safety tiers based on risk:

- **Tier 1 (Safe Read-Only):** list_block_devices, list_loop_devices, detect_filesystem, list_mounts, list_files. These execute automatically.
- **Tier 2 (Controlled Creation / Cleanup):** Read-only loop attach, read-only mount under /mnt/agent, unmount and detach of the agent's own devices. These execute automatically with the result shown in the terminal.
- **Tier 3 (Write Access):** Attaching or mounting an image *read-write*. The tool halts and prompts `[CONFIRM] ... [y/N]` on the terminal; anything other than an explicit "y" is treated as a refusal and the model is told the user declined.

B. Input Validation and No-Shell Execution

No call uses `shell=True`; every subprocess is a fixed argument list, so shell injection such as `; reboot` has no effect. Device paths must match `/dev/[A-Za-z0-9/_-]+` and, by default, `/dev/loopN(pM)` only. Mount names must match `[A-Za-z0-9._-]{1,64}`, which blocks traversal through the name.

C. Kernel-Level Mount Sandboxing

- **MS_RDONLY (1):** Mounts pass this flag by default via libc `mount()`, so the image cannot be modified accidentally or maliciously.
- **Mountpoint sandboxing:** All mounts are created only under `/mnt/agent/<name>` (MOUNT_ROOT).
- **Path confinement:** `list_files` resolves symlinks and rejects any path that escapes the mount point.
- **Real-disk protection:** Non-loop devices such as /dev/sda are blocked unless the operator starts the agent with `--allow-real-disks`.

Threat Vector / Risky Prompt	Operating System Risk	Guardrail Implemented in Project
"Mount /dev/sda1 and show me the files"	Accidental access to the host disk	device_allowed() accepts only /dev/loopN unless --allow-real-disks is passed.
"Mount the ISO read-write and edit it"	Unintended modification of an image	Read-only is the default; read-write triggers the [y/N] confirmation gate.
Shell token injection (; reboot)	Arbitrary command execution	All subprocess calls pass explicit argv lists; no shell string is ever evaluated.
Mount name such as ../../etc	Mounting outside the sandbox	mount_dir() enforces a strict name regex and joins only under /mnt/agent.
Path traversal in list_files (../../etc/shadow)	Unauthorized file access and leak	Paths are canonicalized with resolve() and rejected if outside the mount root.
Model calls an undefined tool or passes bad arguments	Crash or undefined behavior	Dispatcher wraps every call in try/except and returns the error to the model.

Chapter 4: Implementation Methodology

Chapter 4 presents our implementation methodology, technical tool stack and the actual codebase developed during this review cycle. The complete source of `mount_agent.py` is shown below, divided according to the implementation steps.

Development Environment & Tech Stack

We developed the agent on a Linux system, since losetup and mount(2) are Linux-specific. The program checks for Linux and root privileges at startup and exits with a clear message otherwise.

Component	Technology / Package	Role in Student Project
Language	Python 3.10+	Core implementation language; talks to the C library through ctypes.
LLM SDK	google-genai (>= 0.8.0)	Gemini client used for function calling (tool use).
LLM Model	gemini-2.5-flash	Interprets natural language and selects tools; temperature 0 for repeatable behavior.
System Tools	util-linux: lsblk, losetup, blkid, udevadm	Device listing, loop attachment, filesystem probing, waiting for partition nodes.
Kernel Interface	libc mount() / umount2() via ctypes	Direct system calls with the MS_RDONLY flag instead of shelling out to mount.
Setup	setup.sh, requirements.txt	Checks Python version, installs dependencies, verifies tools and GEMINI_API_KEY.

Step-by-Step Implementation

Step 1: Configuration, libc Bindings and Safety Helpers

In Step 1, we defined the model name, the sandbox root (/mnt/agent) and the read-only flag, bound libc `mount` and `umount2` through ctypes, and wrote the helpers every tool relies on: a no-shell command runner, the loop-device test, the device allow-list, the mount-name validator and the confirmation prompt.

```
#!/usr/bin/env python3
"""
Mount & Partition Agent
Natural language -> lsblk / losetup / mount(2) via Gemini tool use.

Safety model (the LLM never gets a shell):
    * No shell=True anywhere; every command is a fixed argv list.
    * Mounts only land under MOUNT_ROOT (/mnt/agent/<name>).
    * Read-only by default; read-write requires explicit human confirmation.
    * Only loop devices can be attached/mounted/detached unless --allow-real-disks.
    * File browsing is confined to the mount point (symlinks resolved).
"""

import argparse
import ctypes
import ctypes.util
import json
import os
import re
import subprocess
import sys
from pathlib import Path

from google import genai
from google.genai import types

# ===== Configuration =====
MODEL = "gemini-2.5-flash"
MOUNT_ROOT = Path("/mnt/agent")
MS_RDONLY = 1
ALLOW_REAL_DISKS = False

# ===== libc bindings for mount(2) / umount2(2) =====
libc = ctypes.CDLL(ctypes.util.find_library("c"), use_errno=True)
libc.mount.argtypes = [
    ctypes.c_char_p, ctypes.c_char_p,
    ctypes.c_char_p, ctypes.c_ulong, ctypes.c_char_p,
]
libc.umount2.argtypes = [ctypes.c_char_p, ctypes.c_int]


# ===== Helpers =====
```

```
def run(argv: list[str], timeout: int = 30) -> dict:
    p = subprocess.run(argv, capture_output=True, text=True, timeout=timeout)
    return {
        "ok": p.returncode == 0,
        "stdout": p.stdout.strip(),
        "stderr": p.stderr.strip(),
    }


def err(msg: str) -> dict:
    return {"ok": False, "error": msg}


def is_loop(dev: str) -> bool:
    return re.fullmatch(r"/dev/loop\d+(p\d+)?", dev) is not None


def device_allowed(dev: str) -> bool:
    if not re.fullmatch(r"/dev/[A-Za-z0-9/_-]+", dev):
        return False
    return is_loop(dev) or ALLOW_REAL_DISKS


def mount_dir(name: str) -> Path | None:
    if not re.fullmatch(r"[A-Za-z0-9._-]{1,64}", name) or name in (".", ".."):
        return None
    return MOUNT_ROOT / name
def confirm(question: str) -> bool:
    return input(f"\n[CONFIRM] {question} [y/N] ").strip().lower() == "y"
```

Step 2: Implementing the Safe Tool Functions

In Step 2, we implemented the nine tools. Each one validates its own arguments, returns a JSON-serializable dictionary and never raises to the model. The write-access paths of `attach_loop` and `mount_device` are guarded by the confirmation prompt.

```
# ===== Tool implementations =====
def list_block_devices() -> dict:
    r = run(["lsblk", "-J", "-o",
             "NAME,PATH,SIZE,TYPE,FSTYPE,LABEL,RO,MOUNTPOINTS,PKNAME"])
    return json.loads(r["stdout"]) if r["ok"] else err(r["stderr"])


def list_loop_devices() -> dict:
    r = run(["losetup", "-J", "-l"])
    if not r["ok"]:
        return err(r["stderr"])
    return json.loads(r["stdout"]) if r["stdout"] else {"loopdevices": []}
def attach_loop(image_path: str, read_only: bool = True) -> dict:
    """Attach an image to a free loop device; read-write needs human confirmation."""
    p = Path(image_path).expanduser().resolve()
    if not p.is_file():
        return err(f"{p} is not a regular file")
    if not read_only and not confirm(f"Attach {p} READ-WRITE?"):
        return err("User declined read-write attach")
    argv = ["losetup", "--find", "--show", "--partscan"]
    if read_only:
        argv.append("--read-only")
    r = run(argv + [str(p)])
    if not r["ok"]:
        return err(r["stderr"])
    dev = r["stdout"]
    run(["udevadm", "settle"])
    return {
        "ok": True,
        "device": dev,
        "hint": "Call list_block_devices to see partitions (e.g. loop0p1).",
    }


def detect_filesystem(device: str) -> dict:
    """Probe a device with blkid; failure may mean it holds a partition table."""
    if not device_allowed(device):
        return err(f"{device} not permitted")
    r = run(["blkid", "-p", "-o", "export", device])
    if not r["ok"]:
```

```
        return err(r["stderr"] or "no recognizable filesystem (partition table?)")
    return {
        "ok": True,
        **dict(line.split("=", 1) for line in r["stdout"].splitlines() if "=" in line),
    }


def mount_device(device: str, name: str,
                 fstype: str | None = None, read_only: bool = True) -> dict:
    """Mount at /mnt/agent/<name>; read-only by default, fstype auto-detected."""
    if not device_allowed(device):
        return err(f"{device} not permitted (loop devices only by default)")
    target = mount_dir(name)
    if target is None:
        return err("name must match [A-Za-z0-9._-]{1,64}")
    if not read_only and not confirm(f"Mount {device} READ-WRITE at {target}?"):
        return err("User declined read-write mount")
    if not fstype:
        det = detect_filesystem(device)
        if not det.get("ok"):
            return det
        fstype = det.get("TYPE")
        if not fstype:
            return err("could not determine filesystem type")
    target.mkdir(parents=True, exist_ok=True)
    flags = MS_RDONLY if read_only else 0
    rc = libc.mount(
        device.encode(), str(target).encode(), fstype.encode(), flags, None
    )
    if rc != 0:
        e = ctypes.get_errno()
        try:
            target.rmdir()
        except OSError:
            pass
        return err(f"mount failed: {os.strerror(e)} (errno {e})")
    return {
        "ok": True,
        "mountpoint": str(target),
        "fstype": fstype,
        "read_only": read_only,
    }
def list_mounts() -> dict:
    out = []
    for line in Path("/proc/self/mountinfo").read_text().splitlines():
        f = line.split()
        mp = f[4].replace("\\040", " ")
        if mp.startswith(str(MOUNT_ROOT) + "/"):
            sep = f.index("-")
            out.append({
                "mountpoint": mp,
                "fstype": f[sep + 1],
                "source": f[sep + 2],
                "options": f[5],
            })
    return {"mounts": out}


def unmount(name: str, detach_loop: bool = True) -> dict:
    """Unmount, remove the directory and optionally detach the backing loop device."""
    target = mount_dir(name)
    if target is None or not target.exists():
        return err(f"no mount named '{name}'")
    source = next(
        (m["source"] for m in list_mounts()["mounts"]
         if m["mountpoint"] == str(target)),
        None,
    )
    if libc.umount2(str(target).encode(), 0) != 0:
        e = ctypes.get_errno()
        return err(f"umount failed: {os.strerror(e)} (is it busy?)")
    try:
        target.rmdir()
    except OSError:
        pass
    res = {"ok": True, "unmounted": str(target)}
    if detach_loop and source and is_loop(source):
        base = re.sub(r"p\d+$", "", source)
        res["detach"] = run(["losetup", "-d", base])
    return res
```

```
def detach_loop_device(device: str) -> dict:
    if not is_loop(device):
        return err("only /dev/loopN devices can be detached")
    r = run(["losetup", "-d", device])
    return {"ok": r["ok"], "stderr": r["stderr"]}


def list_files(name: str, subpath: str = "", max_entries: int = 200) -> dict:
    """List a directory inside a mount; browsing is confined to the mount point."""
    target = mount_dir(name)
    if target is None or not target.is_dir():
        return err(f"no mount named '{name}'")
    root = target.resolve()
    p = (root / subpath.lstrip("/")).resolve()
    if root != p and root not in p.parents:
        return err("path escapes the mount point")
    if not p.is_dir():
        return err(f"'{subpath or '/'}' is not a directory")
    entries = []
    for i, child in enumerate(sorted(p.iterdir())):
        if i >= max_entries:
            entries.append("... (truncated)")
            break
        kind = "d" if child.is_dir() else "l" if child.is_symlink() else "f"
        size = child.stat().st_size if kind == "f" else None
        entries.append({"name": child.name, "type": kind, "size": size})
    return {"ok": True, "path": str(p), "entries": entries}
```

Step 3: Tool Registry, Function Declarations and System Prompt

In Step 3, we registered each function in `TOOLS_IMPL` and described it to Gemini with a typed `FunctionDeclaration`. The system prompt fixes the standard workflow (attach, inspect partitions, detect filesystem, mount, list files) and instructs the model to default to read-only and clean up after failures.

```
# ===== Tool registry & Gemini function declarations =====
TOOLS_IMPL = {
    "list_block_devices": list_block_devices,
    "list_loop_devices": list_loop_devices,
    "attach_loop": attach_loop,
    "detect_filesystem": detect_filesystem,
    "mount_device": mount_device,
    "list_mounts": list_mounts,
    "unmount": unmount,
    "detach_loop_device": detach_loop_device,
    "list_files": list_files,
}

TOOLS = [
    types.Tool(function_declarations=[
        types.FunctionDeclaration(
            name="list_block_devices",
            description=(
                "Run lsblk and return all block devices, partitions, "
                "filesystems and mountpoints as JSON."
            ),
            parameters=types.Schema(type=types.Type.OBJECT, properties={}),
        ),
        types.FunctionDeclaration(
            name="list_loop_devices",
            description=(
                "Run losetup -l: lists which loop devices exist "
                "and which image files back them."
            ),
            parameters=types.Schema(type=types.Type.OBJECT, properties={}),
        ),
        types.FunctionDeclaration(
            name="attach_loop",
            description=(
                "Attach a disk image or ISO file to a free loop device "
                "(losetup --find --partscan). Returns the loop device path. "
                "Read-only by default."
            ),
            parameters=types.Schema(
                type=types.Type.OBJECT,
                properties={
                    "image_path": types.Schema(
```

```
                    type=types.Type.STRING,
                    description="Absolute or ~ path to the image file.",
                ),
                "read_only": types.Schema(
                    type=types.Type.BOOLEAN,
                    description="Attach read-only (default: true).",
                ),
            },
            required=["image_path"],
        ),
    ),
    types.FunctionDeclaration(
        name="detect_filesystem",
        description=(
            "Probe a device or partition with blkid and return its TYPE "
            "(iso9660, ext4, vfat, ...), LABEL, UUID. If it fails, the "
            "device may hold a partition table: call list_block_devices "
            "to find its partitions."
        ),
        parameters=types.Schema(
            type=types.Type.OBJECT,
            properties={
                "device": types.Schema(
                    type=types.Type.STRING,
                    description="Block device path, e.g. /dev/loop0.",
                ),
            },
            required=["device"],
        ),
    ),
    types.FunctionDeclaration(
        name="mount_device",
        description=(
            "mount(2) a device/partition at /mnt/agent/<name>. "
            "fstype is auto-detected if omitted. Read-only by default."
        ),
        parameters=types.Schema(
            type=types.Type.OBJECT,
            properties={
                "device": types.Schema(
                    type=types.Type.STRING,
                    description="Block device path, e.g. /dev/loop0 or /dev/loop0p1.",
                ),
                "name": types.Schema(
                    type=types.Type.STRING,
                    description="Short directory name, e.g. 'ubuntu-iso'.",
                ),
                "fstype": types.Schema(
                    type=types.Type.STRING,
                    description="Filesystem type; auto-detected if omitted.",
                ),
                "read_only": types.Schema(
                    type=types.Type.BOOLEAN,
                    description="Mount read-only (default: true).",
                ),
            },
            required=["device", "name"],
        ),
    ),
    types.FunctionDeclaration(
        name="list_mounts",
        description="List all mounts this agent has made under /mnt/agent.",
        parameters=types.Schema(type=types.Type.OBJECT, properties={}),
    ),
    types.FunctionDeclaration(
        name="unmount",
        description=(
            "Unmount /mnt/agent/<name>, remove the directory, "
            "and detach its backing loop device."
        ),
        parameters=types.Schema(
            type=types.Type.OBJECT,
            properties={
                "name": types.Schema(
                    type=types.Type.STRING,
                    description="The mount name given when mount_device was called.",
                ),
                "detach_loop": types.Schema(
                    type=types.Type.BOOLEAN,
                    description="Also detach the backing loop device (default: true).",
                ),
            },
            required=["name"],
        ),
```

```
        ),
        types.FunctionDeclaration(
            name="detach_loop_device",
            description=(
                "Detach a loop device with no active mounts "
                "(cleanup after a failed mount attempt)."
            ),
            parameters=types.Schema(
                type=types.Type.OBJECT,
                properties={
                    "device": types.Schema(
                        type=types.Type.STRING,
                        description="Loop device path, e.g. /dev/loop3.",
                    ),
                },
                required=["device"],
            ),
        ),
        types.FunctionDeclaration(
            name="list_files",
            description=(
                "List a directory inside one of the agent's mounts. "
                "Browsing is confined to the mount point."
            ),
            parameters=types.Schema(
                type=types.Type.OBJECT,
                properties={
                    "name": types.Schema(
                        type=types.Type.STRING,
                        description="The mount name.",
                    ),
                    "subpath": types.Schema(
                        type=types.Type.STRING,
                        description="Subdirectory inside the mount to list (default: root).",
                    ),
                    "max_entries": types.Schema(
                        type=types.Type.INTEGER,
                        description="Maximum entries to return (default: 200).",
                    ),
                },
                required=["name"],
            ),
        ),
    ])
]

SYSTEM = """\
You are a Linux storage assistant that mounts disk images and partitions for the user.

Standard workflow for an image file:
1. attach_loop (read-only by default)
2. list_block_devices — check whether the image has partitions
3. detect_filesystem on the right node (the loop device itself for ISOs)
4. mount_device — mounts it at /mnt/agent/<name>
5. list_files — show the user what's inside
For ISOs the loop device itself is usually iso9660.
For full disk images, mount the relevant partition (ask the user which if several are plausible).

Always default to read-only. Clean up after failures (detach loop devices you attached).
Be concise and report the mountpoint once the mount succeeds.
"""
```

Step 4: The Agentic Loop and Entry Point

In Step 4, we built the REPL and the agentic loop. After each user message the loop calls Gemini, executes every returned function call, feeds the results back, and repeats until the model answers in plain text. The entry point parses `--allow-real-disks` and enforces the Linux and root checks.

```
# ===== Agent loop =====
def chat() -> None:
    api_key = os.environ.get("GEMINI_API_KEY")
    if not api_key:
        sys.exit("GEMINI_API_KEY not set. Export it before running with sudo -E.")

    client = genai.Client(api_key=api_key)
    history: list[types.Content] = []

    print("Mount & Partition Agent  (Ctrl-D or Ctrl-C to quit)")
    print(f"  Model      : {MODEL}")
    print(f"  Mount root : {MOUNT_ROOT}")
    print(f"  Real disks : {'allowed' if ALLOW_REAL_DISKS else 'blocked (pass --allow-real-disks)'}")
    print()
```

```
    while True:
        try:
            user_text = input("> ").strip()
        except (EOFError, KeyboardInterrupt):
            print("\nBye.")
            return
        if not user_text:
            continue

        history.append(types.Content(role="user", parts=[types.Part(text=user_text)]))

        # Agentic loop: keep going until the model stops calling tools
        while True:
            response = client.models.generate_content(
                model=MODEL,
                contents=history,
                config=types.GenerateContentConfig(
                    system_instruction=SYSTEM,
                    tools=TOOLS,
                    temperature=0,
                ),
            )

            candidate = response.candidates[0]
            history.append(types.Content(role="model", parts=candidate.content.parts))

            tool_calls = [
                p for p in candidate.content.parts
                if p.function_call is not None
            ]

            for part in candidate.content.parts:
                if part.text and part.text.strip():
                    print(part.text)

            if not tool_calls:
                break

            function_responses = []
            for part in tool_calls:
                fc = part.function_call
                print(f"  [tool] {fc.name}({json.dumps(dict(fc.args))})")
                try:
                    result = TOOLS_IMPL[fc.name](**dict(fc.args))
                except Exception as exc:
                    result = err(f"{type(exc).__name__}: {exc}")
                function_responses.append(
                    types.Part(
                        function_response=types.FunctionResponse(
                            name=fc.name,
                            response=result,
                        )
                    )
                )
            history.append(
                types.Content(role="user", parts=function_responses)
            )


# ===== Entry point =====
if __name__ == "__main__":
    ap = argparse.ArgumentParser(
        description="Natural-language Linux mount/partition agent powered by Gemini."
    )
    ap.add_argument(
        "--allow-real-disks",
        action="store_true",
        help="Permit mounting non-loop block devices (/dev/sdX, nvme...)",
    )
    args = ap.parse_args()
    ALLOW_REAL_DISKS = args.allow_real_disks

    if sys.platform != "linux":
        sys.exit("This agent requires Linux (needs losetup and mount(2)).")
    if os.geteuid() != 0:
        sys.exit("Run as root:  sudo -E python3 mount_agent.py")

    MOUNT_ROOT.mkdir(parents=True, exist_ok=True)
    chat()
```

Demonstration & Verification Scenarios

The scenarios prepared in `demo.sh` exercise every part of the workflow. Each prompt is typed at the REPL started with `sudo -E python3 mount_agent.py`:

Scenario	Example Prompt	Expected Tool Sequence
1. Mount an ISO	Mount ~/Downloads/ubuntu-22.04.iso so I can explore the files	attach_loop → detect_filesystem (iso9660) → mount_device → list_files
2. Check mounts	What's mounted right now?	list_mounts
3. Browse a subfolder	List the files inside the casper folder	list_files (subpath = casper)
4. Partitioned image	Attach debian.img and show me its partitions	attach_loop → list_block_devices → detect_filesystem per partition → ask user which to mount
5. Clean up	Unmount everything and detach all loop devices	list_mounts → unmount (detach_loop = true) for each mount